Model Inversion Attacks and Privacy Defences on Face Recognition Neural Networks

Aya Mouffok Mohamed Al Amin Saad Ines Sekfali Malak Meliani Hanaa Chahbar

Neural networks that are trained on private images can leak that data through their parameters. This paper studies a threat called a *Model Inversion (MI) Attack*, where an attacker uses only the weights of a trained model to rebuild the original training images. We use the AT&T/ORL face database, which has 400 images of 40 people. First, we build a CNN that is designed to memorize all its training images on purpose. It reaches 100% training accuracy and 94.5% validation accuracy. Then we run a gradient-ascent attack to rebuild all 40 faces. The attack reaches a mean SSIM of 0.2458, which is 26.4 times higher than our random-noise baseline of 0.0093. This confirms that real face information leaks from the model. Next, we design a defense. It combines Differentially Private SGD (DP-SGD) with five extra techniques: MC Dropout, Label Smoothing, Data Augmentation, Batch Normalization, and Global Average Pooling. With the defense, the attack SSIM drops to 0.0319, which is only 3.4 times the baseline. The attack no longer works. The defended model keeps 65.5% validation accuracy. The formal privacy budget is $(\varepsilon = 20.02,\ \delta = 2.94 \times 10^{-3})$-DP. A test with three different random seeds confirms that the defense results are stable and not a coincidence.

***Index Terms*—Model Inversion Attack, Differential Privacy, DP-SGD, Face Recognition, CNN, Gradient Ascent, SSIM, Privacy, ENSIA**

I. Introduction

A. The Problem

When a neural network learns from private data, it stores patterns from that data inside its weights. This is useful for making predictions. But it also creates a risk: someone who has access to the model weights may be able to get back the original private data.

Imagine a model f_θ that is trained to recognise $K = 40$ different people from face images. Here, θ means all the learned weights of the model. After training, the model is used on a server. The original training images are kept private and are not shared. But the model weights θ can sometimes be read by an outside person.

The main question of this paper is:

Can an attacker who only has the model weights θ rebuild the private face images that were used during training?

The answer is yes. This attack is called a **Model Inversion (MI) Attack** [1]. It works because the model encodes specific patterns of the training images inside its parameters during training.

B. Why This Matters

This problem is important in many real situations:

- **Healthcare:** A hospital trains a model on patient X-ray images. An attacker can try to rebuild those images from the model.
- **Finance:** A bank trains a fraud-detection model on customer data. An attacker can try to rebuild sensitive transaction records.
- **Biometrics:** A company uses a face-recognition model. An attacker can try to rebuild employee or customer faces.

Laws like the EU GDPR, the US HIPAA, and China's PIPL say that AI systems must not leak personal data. This project shows exactly when they do leak data, and how to stop it.

C. What This Paper Does

This paper makes the following contributions:

1) We show a working gradient-ascent MI attack on a face-recognition CNN.
2) We compute a random-noise baseline so that SSIM and MSE results can be correctly understood.
3) We test 9 different combinations of attack settings to find the best ones.
4) We design a two-phase defence that uses DP-SGD and five extra techniques.
5) We compute the formal privacy budget using the Rényi DP accountant.
6) We run the attack with three different random seeds to check that results are stable.

The paper is organised as follows. Section II describes the dataset, the model, and the setup. Section III explains the MI attack. Section IV explains the defence. Section V shows the results. Section VI discusses the limits of this work. Section VII gives the conclusion.

II. Dataset, Model, and Setup

A. The AT&T/ORL Face Database

All experiments use the AT&T/ORL face database [4]. Table I shows its main properties.

Each image is stored as a vector with 4096 numbers, because $4096 = 64 \times 64$. The neural network learns a function:

$$f_\theta : \mathbb{R}^{4096} \to \{1, \ldots, 40\} \tag{1}$$

This function takes an image and outputs one of 40 identity labels.

TABLE I
AT&T/ORL DATABASE PROPERTIES

Property	Value	What it means
Total images	400	Full dataset
People (classes)	40	One class per person
Images per person	10	Different conditions
Original size	92×112 px	Grayscale PGM
Resized to	64×64 px	Uniform input size
Training images	200 (5 per person)	Causes memorisation
Test images	200 (5 per person)	Used for evaluation

An important point: with only 5 training images per person, the model cannot learn a general idea of what a face looks like. It can only memorise those 5 specific images. This memorisation is what makes the MI attack possible.

B. Making the Experiment Reproducible

The experiment has several steps that use random numbers. These include splitting the data, starting the attack from random noise, adding DP noise, and starting the model weights. If we do not control these random numbers, the results will be different every time we run the code.

To fix this, we set a global seed $\texttt{SEED} = 42$ for all random number generators: Python's built-in `random`, NumPy, and PyTorch. We also turn on deterministic GPU mode:

```
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

This makes sure the results are exactly the same every time.

C. How We Split the Data

A simple random split of 400 images into training and test sets is not good in this case. If two very similar images of the same person go into both sets, the model might score well on the test set just because it memorised the training images. This would make our evaluation wrong.

Instead, we use a **subject-stratified split**. For each person, we randomly pick 5 images for training and keep the other 5 for testing. This is done with the fixed seed so it is reproducible. For example:

$$\underbrace{[i_1, i_2, i_3, i_4, i_5]}_{\text{TRAIN}} \mid \underbrace{[i_6, i_7, i_8, i_9, i_{10}]}_{\text{TEST}}$$

This follows the method used in the original ORL paper [4].

D. Image Preprocessing

Before giving images to the network, we apply these three steps:

1) **Grayscale:** Keep only one colour channel.
2) **Resize** to 64×64 pixels: Make all images the same size.
3) **ToTensor:** Change pixel values from integers $[0, 255]$ to floats $[0, 1]$.

We do *not* use standard normalisation (subtracting the mean or scaling by standard deviation). Keeping pixels in $[0, 1]$ is important because the attack reconstructs images, and we need to display them directly as grayscale images without any extra conversion step.

E. The Vulnerable Model: Built to Memorise

The goal of this model is not to generalise well. The goal is to **memorise the training images as much as possible**. Each design choice is made on purpose to increase memorisation.

1) Large Convolutional Layers: 64 → 128 → 256 Channels: Convolutional layers find features in the image, such as edges or textures, by applying learned filters. More channels mean the model can store more specific details. We use 64, 128, and 256 channels, which is larger than a typical model. This gives the model more space to store the exact details of each training face.

2) Average Pooling Instead of Max Pooling: Pooling layers reduce the size of feature maps. Max pooling keeps only the largest value in each small area, which throws away about 75% of the information. Average pooling keeps more information by computing the mean:

$$\text{AvgPool}(x)_{i,j} = \frac{1}{|P|} \sum_{(m,n) \in P} x_{m,n} \tag{2}$$

Average pooling keeps more detail, which the attacker can later use to rebuild the image.

3) Large Fully Connected Layer: 1024 Neurons: To classify 40 people, the minimum number of neurons needed is about 256. We use 1024 neurons instead. This creates a large memory space where the model stores very specific information about each training face. More neurons mean more memorisation.

4) LeakyReLU Activation: Standard ReLU sets all negative values to zero:

$$\text{ReLU}(x) = \max(0, x) \tag{3}$$

When a neuron outputs zero, its gradient is also zero. This stops the MI attack from sending information back through that neuron to the input pixels.

LeakyReLU fixes this by keeping a small value for negative inputs:

$$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ 0.1x & x \leq 0 \end{cases} \tag{4}$$

With LeakyReLU, gradients always flow back to the input. This makes the attack work better because the attacker always gets a useful gradient signal.

5) No Regularisation: Normal training uses tools like weight decay or dropout to stop the model from memorising too much. We do not use any of these:

- **Weight decay = 0:** The model weights can grow as large as they need to perfectly encode each training image.
- **No dropout:** All neurons are always active, so the model is as consistent as possible.

6) Training Until 100% Accuracy: We train the model for 60 epochs. When training accuracy reaches 100%, it means the model has memorised every training image. The model also reaches 94.5% validation accuracy, which shows that what it memorised is real facial information, not random noise. This is important because the MI attack can only recover useful information if the model has learned real features.

III. The Model Inversion Attack

A. What the Attack Does

The MI attack treats image reconstruction as an optimisation problem. We want to find an input image x^* that makes the model output a high probability for the target person y_t, while also looking smooth:

$$x^* = \arg\max_{x \in [0,1]^d} \; p_\theta(y_t \mid x) - \lambda_{\mathrm{TV}} \mathcal{L}_{\mathrm{TV}}(x) \tag{5}$$

where $d = 4096$ is the number of pixels, $p_\theta(y_t \mid x)$ is the model's confidence for the target class, and $\mathcal{L}_{\mathrm{TV}}$ is a smoothness term explained below.

B. How We Measure the Attack

1) Mean Squared Error (MSE): MSE measures the average pixel difference between the rebuilt image and the real image:

$$\mathrm{MSE}(x_{\mathrm{true}}, x_{\mathrm{pred}}) = \frac{1}{N} \sum_{i=1}^{N} \big(x_{\mathrm{true},i} - x_{\mathrm{pred},i}\big)^2 \tag{6}$$

where $N = 4096$. Values range from 0 (identical images) to 1 (completely different). A lower MSE means the reconstruction is better.

2) Structural Similarity Index (SSIM): SSIM measures how similar two images look to a human. It compares the brightness, contrast, and structure of the two images [5]:

$$\mathrm{SSIM}(x, y) = [l(x,y)]^\alpha \cdot [c(x,y)]^\beta \cdot [s(x,y)]^\gamma \tag{7}$$

Values range from 0 (no similarity) to 1 (perfect match). A higher SSIM means the reconstruction looks more like the real image.

3) The Random-Noise Baseline: A single SSIM number tells us nothing by itself. We need something to compare it to. We compute a **random-noise baseline**: we generate 50 images of pure random pixel noise per person, and compute the average SSIM and MSE of those noise images compared to the real faces:

$$\mathrm{SSIM}_{\mathrm{baseline}} = \mathbb{E}_{x_{\mathrm{rand}} \sim \mathcal{U}[0,1]^d} \big[\mathrm{SSIM}(x_{\mathrm{rand}}, x_{\mathrm{true}})\big] \tag{8}$$

This gives $\mathrm{SSIM}_{\mathrm{baseline}} = 0.0093$ and $\mathrm{MSE}_{\mathrm{baseline}} = 0.1226$. Any attack that reaches more than $4 \times 0.0093 = 0.039$ SSIM is showing real leakage, not just random variation.

C. The Gradient Ascent Algorithm

The attack starts from a random noise image $x^{(0)}$ and updates it step by step to make the model more confident about the target person. The update rule is:

$$x^{(t+1)} = \mathrm{clip}\Big(x^{(t)} - \alpha \nabla_x \mathcal{L}_{\mathrm{total}}\big(x^{(t)}\big),\ 0,\ 1\Big) \tag{9}$$

where α is the learning rate (step size), the clip keeps pixels in $[0, 1]$, and the total loss is:

$$\mathcal{L}_{\mathrm{total}}(x) = \mathcal{L}_{\mathrm{CE}}(f_\theta(x), y_t) + \lambda_{\mathrm{TV}} \mathcal{L}_{\mathrm{TV}}(x) \tag{10}$$

Here $\mathcal{L}_{\mathrm{CE}}$ is the cross-entropy loss for the target class. We minimise it, which is the same as maximising the model's confidence for y_t.

The full steps are in Algorithm 1.

Algorithm 1 Gradient Ascent Model Inversion Attack

Require: Model f_θ, target class y_t, learning rate α, TV weight λ_{TV}, steps $T = 500$

Freeze all model weights (no weight update during attack)
Start with random noise: $x^{(0)} \sim \mathcal{N}(0,1)^d$, enable gradients on x
for $t = 0, 1, \ldots, T-1$ **do**
  Run image through model: $\hat{y} = f_\theta(x^{(t)})$
  Compute $\mathcal{L}_{\mathrm{CE}} = -\log p_\theta(y_t \mid x^{(t)})$
  Compute $\mathcal{L}_{\mathrm{TV}}(x^{(t)})$ (see Eq. 11)
  $\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{CE}} + \lambda_{\mathrm{TV}} \mathcal{L}_{\mathrm{TV}}$
  Compute gradient: $g \leftarrow \nabla_x \mathcal{L}_{\mathrm{total}}$
  Update pixels: $x^{(t+1)} \leftarrow \mathrm{clip}(x^{(t)} - \alpha g,\ 0,\ 1)$
  Reset gradient on x to zero
end for
return Rebuilt image $x^{(T)}$ and confidence $p_\theta(y_t \mid x^{(T)})$

1) Total Variation Regularisation: Without any smoothness control, gradient ascent often produces noisy images where nearby pixels have very different values. This makes the image hard to read visually. We add a **Total Variation (TV)** term to the loss. It penalises large differences between neighbouring pixels:

$$\mathcal{L}_{\mathrm{TV}}(x) = \sum_{i,j} \Big(|x_{i+1,j} - x_{i,j}| + |x_{i,j+1} - x_{i,j}|\Big) \tag{11}$$

The weight λ_{TV} controls how much we care about smoothness. A small value gives more confidence but a noisy image. A large value gives a smooth but blurry image.

D. Finding the Best Attack Settings

We cannot just guess good values for α and λ_{TV}. We test all 9 combinations from a 3×3 grid:

$$\alpha \in \{0.01,\ 0.05,\ 0.10\}, \quad \lambda_{\mathrm{TV}} \in \{10^{-5},\ 10^{-4},\ 10^{-3}\}$$

Each combination is tested on 5 people with 300 attack steps. We pick the one with the highest mean SSIM. The main results are shown in Table II.

TABLE II
Hyperparameter Search: Mean SSIM (5 people, 300 steps)

TV Weight (λ_{TV})	Learning Rate (α)	Mean SSIM
10^{-5}	0.10	0.1452
10^{-4}	0.10	0.2730
10^{-3}	0.01	0.2129
10^{-3}	0.05	**0.3012**

The best setting is $\lambda_{\mathrm{TV}} = 10^{-3}$ and $\alpha = 0.05$. It reaches mean SSIM = 0.3012, which is about 32 times the random baseline. We use these settings for all further attacks.

E. Attack on All 40 People

We now run the attack on all 40 people using the best settings and 500 steps. The results are:

- **Mean Confidence:** 1.0000 ± 0.0000 — The model is 100% sure about the target person for every class. The attack fully exploits the model.
- **Mean SSIM:** 0.2458 ± 0.0617 — This is 26.4 times the baseline of 0.0093. The rebuilt images look like real faces, not noise.
- **Mean MSE:** 0.0479 ± 0.0135 — Much lower than the baseline MSE of 0.1226. The pixels of the rebuilt image are close to the real image.

We also record how the image changes at steps 0, 50, 100, 200, 300, and 499 for one person. This shows how the image slowly changes from random noise to a face-like picture. We also compare the feature maps (internal layer outputs) of the real image and the rebuilt image. They look very similar, which explains why the model is so confident: it processes both images in almost the same way internally.

IV. The Privacy Defence

A. The Core Problem

We want to stop the model from leaking private face information. But any defence that stops the model from memorising private data will also make it less accurate. This is called the **privacy–utility trade-off**.

A simple solution would be to add a lot of noise during training. But this does not work here because the dataset is small ($N = 200$ training images). With too little noise, there is no privacy. With too much noise, the model cannot learn anything useful.

We solve this with a two-phase training strategy.

B. Phase 1: Normal Training (No Privacy)

We train the full CNN for 80 epochs using normal SGD. The model learns general things about faces, like edges, shapes, and textures. It reaches about 82% validation accuracy. No privacy mechanisms are used in this phase.

C. Phase 2: Private Fine-Tuning with DP-SGD

In Phase 2, we **freeze** the first two convolutional layers (`conv1` and `conv2`). Their weights do not change. We only update the deeper layers (`conv3` and the fully connected layers) using DP-SGD. The deeper layers store identity-specific information, so this is where privacy protection is most important.

1) How DP-SGD Works: DP-SGD [2] changes the normal gradient update in two steps.

Step 1 — Clip each gradient:

$$\tilde{g}_i = g_i \cdot \min\left(1,\ \frac{C}{\|g_i\|_2}\right) \tag{12}$$

Each training sample has its own gradient g_i. We clip it so that its size (its ℓ_2 norm) is at most $C = 1.0$. This limits how much any one training sample can change the model weights.

Step 2 — Add noise to the average gradient:

$$\bar{g} = \frac{1}{B}\left(\sum_{i=1}^{B} \tilde{g}_i + \mathcal{N}\big(0,\ \sigma^2 C^2 \mathbf{I}\big)\right) \tag{13}$$

We add Gaussian noise with $\sigma = 0.4$ to the sum of clipped gradients, then divide by the batch size $B = 32$. This noise hides the contribution of individual training samples, making it hard to recover them from the model weights.

The parameters we chose:

- $\sigma = 0.4$: Tested to reduce the attack SSIM to near the random baseline.
- $C = 1.0$: A standard clip norm that keeps the sensitivity bounded.
- $B = 32$: A batch size that works well with the dataset size.

2) Stopping DP Training at the Right Time: Long DP training causes a problem: early epochs reach about 64% accuracy, but later epochs drop back to about 58% because the accumulated noise pushes the weights away from their best values. To stop this, we use:

- A maximum of 15 DP training epochs.
- Early stopping with patience 5 (we stop if accuracy does not improve for 5 epochs).
- We save the best checkpoint and use it at the end.

D. Five Extra Defence Techniques

DP-SGD gives a formal privacy guarantee but is not enough on its own for small datasets. We add five more defences.

1) MC Dropout ($p = 0.30$):

$$y = f(x;\, \theta,\, m), \quad m \sim \text{Bernoulli}(p = 0.30) \tag{14}$$

Dropout randomly turns off neurons during each forward pass. We keep dropout active during inference, not just training. This means every time the attacker queries the model, it gives a slightly different gradient. The attack cannot follow a stable gradient path to rebuild the image. The rebuilt image stays noisy.

2) Label Smoothing ($\varepsilon_{\text{LS}} = 0.10$):

$$y_{\text{smooth}} = (1 - \varepsilon_{\text{LS}}) \cdot y_{\text{one-hot}} + \frac{\varepsilon_{\text{LS}}}{K - 1} \tag{15}$$

where $K = 40$. Instead of training the model to output a probability of 1.0 for the correct class and 0.0 for all others, we use softer targets. This reduces overconfidence. The model does not learn to produce very sharp, strong gradient signals, which makes the attack harder to use.

3) Data Augmentation: Each image is randomly changed before being used in training:

$$x' = T(x) \tag{16}$$

where T applies random flips, small rotations, and brightness changes. This stops the model from memorising the exact pixel values of each training image. Instead, the model learns more general face features, which are less useful to an attacker.

4) Batch Normalisation: Batch normalisation adjusts the values inside each layer to have zero mean and unit variance:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \tag{17}$$

This makes training more stable and reduces how much the model reacts to individual training samples.

5) Global Average Pooling (GAP): In the vulnerable model, the 1024-neuron fully connected layer stores detailed spatial information about each face. We replace it with Global Average Pooling, which computes one number per feature map:

$$\mathrm{GAP}(X)_c = \frac{1}{H \cdot W} \sum_{h=1}^{H} \sum_{w=1}^{W} X_{c,h,w} \tag{18}$$

This throws away the detailed spatial information that the attacker needs to rebuild a good image.

E. How All Six Defences Work Together

The six mechanisms each block a different part of the attack:

- **DP-SGD:** Adds noise to gradients during training, giving a formal privacy guarantee.
- **MC Dropout:** Makes the gradient signal unstable during the attack, so the optimisation does not converge.
- **Label Smoothing:** Reduces the sharpness of the model's output, weakening the gradient signal.
- **Data Augmentation + Batch Norm:** Reduce memorisation of specific pixel patterns.
- **GAP:** Removes the high-capacity memory layer that stores detailed face information.

F. Formal Privacy Budget

We compute the privacy budget using the **Rényi Differential Privacy (RDP)** accountant [3]. DP gives a formal guarantee: a mechanism $\mathcal{M}$ satisfies (ε, δ)-DP if, for any two datasets that differ by one sample, and any output set S:

$$\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta \tag{19}$$

A lower ε means stronger privacy.

With our settings ($\sigma = 0.4$, $C = 1.0$, $B = 32$, $N = 200$, 6 DP epochs, $\delta = N^{-1.1}$), the accountant gives:

$$(\varepsilon,\, \delta)\text{-DP} = (20.02,\ 2.94 \times 10^{-3})\text{-DP} \tag{20}$$

The value $\varepsilon = 20.02$ is higher than the $\varepsilon \leq 10$ that is usually considered "strong" privacy. This is because the dataset is small ($N = 200$), which forces a high sampling rate of $q = B/N = 32/200 = 0.16$. A high sampling rate means the model sees each sample often, which increases the privacy cost. The empirical defence still works well. We report this number honestly rather than hiding it.

V. Results

A. Vulnerable vs. Defended Model

We run the same attack (500 steps, $\alpha = 0.05$, $\lambda_{\mathrm{TV}} = 10^{-3}$) on both models. Table III shows the results.

TABLE III
ATTACK RESULTS: VULNERABLE VS. DEFENDED MODEL

Metric	Vulnerable	Defended	Baseline
Validation Accuracy	94.5%	65.5%	2.5% (random)
Attack Confidence	1.0000	0.9171	—
Mean SSIM	0.2458	0.0319	0.0093
SSIM / Baseline	26.4×	3.4×	1.0×
Mean MSE	0.0479	0.0838	0.1226
Privacy (ε)	∞	20.02	—
DP Noise (σ)	0	0.4	—

Confidence: The attack confidence drops only a little, from 1.0000 to 0.9171. The model still classifies the rebuilt image as the target person. But the MC Dropout makes the gradient inconsistent, so the image the attack builds does not look like a real face even though the model is still confident about it.

SSIM: The defended model reaches SSIM = 0.0319. This is 3.4 times the baseline, which is within the noise range (below the 4× threshold). The vulnerable model reaches 26.4 times the baseline. The defence reduces information leakage by about $26.4/3.4 \approx 7.8$ times as measured by SSIM.

MSE: The defended model's MSE of 0.0838 is much closer to the random baseline of 0.1226. The vulnerable model's MSE of 0.0479 is far below the baseline, showing that its rebuilt images are close to the real faces in pixel terms.

B. How to Read the SSIM Values

Table IV shows what different SSIM values mean in this experiment.

TABLE IV
SSIM INTERPRETATION GUIDE

Situation	SSIM	What it means
Random noise	0.0093	Starting point
Threshold for real leakage	> 0.039	4× baseline
Defended model	0.0319	Near noise; defence works
Vulnerable model	0.2458	26.4×; clear leakage

C. Three-Seed Stability Study

A single attack result could come from a lucky or unlucky random start. To check this, we repeat the attack with three different random seeds: $\{42, 7, 123\}$. Table V shows the results.

TABLE V
3-SEED STUDY: SSIM ACROSS RANDOM SEEDS

Seed	Vulnerable SSIM	Defended SSIM
42	0.1974	0.0319
7	0.1932	0.0334
123	0.1943	0.0408
Mean ± std	$\mathbf{0.1950 \pm 0.0018}$	$\mathbf{0.0354 \pm 0.0039}$

The standard deviations are very small compared to the difference between the two models (0.1950 vs. 0.0354). This confirms that the defence is real and not just due to one lucky random start.

D. The Privacy–Utility Trade-off

Table VI gives the final summary of the trade-off between accuracy and privacy.

The defended model loses 29 percentage points of accuracy in exchange for stopping the attack. Whether this is acceptable depends on the situation. In hospitals or biometric systems, a

TABLE VI
FINAL PRIVACY–UTILITY TRADE-OFF

Model	Accuracy	Privacy (ε)	SSIM	Verdict
Vulnerable	94.5%	∞	0.2458	Face leak
Defended	65.5%	20.02	0.0319	Attack fails

less accurate but private model is better. In other applications, the accuracy loss may be too high.

VI. DISCUSSION AND LIMITS

A. The Main Cause: Small Dataset

The small training set ($N = 200$ images) is the main reason for all the problems in this experiment:

1) **Memorisation:** With only 5 images per person, the model cannot generalise. It memorises the training images.
2) **Weak formal privacy ($\varepsilon = 20.02$):** The high sampling rate $q = 0.16$ makes each DP step expensive in terms of privacy.
3) **Accuracy drop under DP:** The noise from DP destroys the weak gradient signal that comes from a small dataset.

The best way to improve all three problems at once is to use a larger dataset.

B. The Formal Privacy Number Is Honest, Not Hidden

The achieved $\varepsilon = 20.02$ is higher than the $\varepsilon \leq 10$ that is usually called "strong" DP. We do not hide this. The empirical results (SSIM dropping to near-noise) are reported separately from the formal bound, so the reader can see what is guaranteed mathematically and what is only observed in practice.

C. Limits of This Study

This experiment uses a small, clean dataset and a specific model. Results may be different with:

- Larger datasets, where memorisation is lower and DP is cheaper.
- Other data types, like text or tabular data.
- Real production systems, which may have more defences built in.

VII. CONCLUSION

This paper shows a full MI attack and defence pipeline for face-recognition neural networks. The five main findings are:

1) **MI attacks work.** The vulnerable model leaks face information with SSIM = 0.2458, which is 26.4 times above the random-noise baseline of 0.0093.
2) **A random-noise baseline is necessary.** Without it, we cannot say if an SSIM value is meaningful. The ratio to the baseline is what decides the verdict, not the raw number.
3) **Formal DP needs a computed budget.** Just adding noise and calling it "Differential Privacy" is not correct. We must compute (ε, δ) with a proper accountant. Our result is $(\varepsilon = 20.02,\ \delta = 2.94 \times 10^{-3})$-DP.
4) **The privacy–utility trade-off is real.** The defended model loses 29 percentage points of accuracy but stops the attack completely, with SSIM dropping to 0.0319 (3.4× baseline).
5) **Three seeds confirm stability.** The defended model's SSIM across seeds is 0.0354 ± 0.0039, showing the results are stable and not a lucky accident.

For future work, the most important step is to train on a larger dataset. A larger dataset would reduce memorisation, lower the formal ε, and reduce the accuracy drop from DP. Other directions include studying federated learning, where data never leaves local devices, and secure aggregation protocols.

ACKNOWLEDGMENT

The authors would like to thank Dr. Karim Lounis for his guidance during the Computer and Network Security course.

REFERENCES

[1] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proc. 22nd ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pp. 1322–1333, 2015.

[2] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proc. 23rd ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pp. 308–318, 2016.

[3] I. Mironov, "Rényi differential privacy of the Gaussian mechanism," in *Proc. IEEE 30th Computer Security Foundations Symposium (CSF)*, pp. 263–275, 2017.

[4] F. S. Samaria and A. C. Harter, "Parameterisation of a stochastic model for human face identification," in *Proc. 2nd IEEE Workshop on Applications of Computer Vision*, pp. 138–142, 1994.

[5] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[6] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, pp. 1050–1059, 2016.